

SYSTEM DESIGN INTERVIEW

Design a Web Search Engine

Google/Bing-scale indexing, ranking, and query serving

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · The Inverted Index
- 06** · Sharding Strategy
- 07** · Query Pipeline
- 08** · Ranking
- 09** · Indexing Pipeline
- 10** · Caching Layers

11 · Doc Store & Snippet Generation

12 · Failure Modes & Recovery

13 · Design Trade-offs

14 · Scalability & Operations

15 · Interview Playbook

01

Problem Statement

Requirements and scope

Design a web-scale search engine like **Google** or **Bing**. The system must index ~100B web pages, answer free-text queries with sub-200 ms latency at ~100K QPS sustained, and rank results so the most relevant documents surface in the top 10. Crawling itself is out of scope (covered in **14 - Web Crawler**); we focus on the **indexing pipeline** and the **online serving stack**.

Clarifying Questions

Question	Answer
Corpus size?	~100B web pages indexed (English-dominant)
Query volume?	~100K QPS sustained, ~500K QPS peak
Latency target?	p99 < 200 ms end-to-end (search box → 10 blue links)
Freshness?	Daily full rebuild + minute-level real-time inserts for trending
Languages?	Multi-lingual; per-language analyzers; assume English here
Personalisation?	Light: locale, recent history; full personalisation out of scope
Crawler?	Assumed solved — focus on index + serve

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Query	GET /search?q=... → ranked list of 10 results + snippets
Spell correct	Auto-suggest correction; “did you mean...”
Synonyms	Expand query terms (car ↔ automobile)
Snippets	Highlight matched terms with surrounding context
Freshness	Trending content searchable within minutes
Filters	Site:, filetype:, date range; advanced operators

Non-Functional Requirements

Requirement	Target
Availability	99.99% — search is the front door
Query latency	p50 < 80 ms · p99 < 200 ms
Throughput	100K QPS sustained · 500K QPS peak
Index size	~100 TB compressed inverted index (across replicas)
Freshness	Daily batch rebuild + real-time insert path (≤ 5 min)
Recall vs precision	Recall ≥ 90% in candidate gen; top-10 precision is the metric

03

Capacity Estimation

Math for scale

Corpus & Index Size

- Pages indexed: **100 billion**
- Avg indexed bytes per page (terms + positions + tf): **~5 KB** uncompressed
- Raw inverted-index size: $100B \times 5 \text{ KB} = \mathbf{500 \text{ TB}}$
- After block compression (PForDelta + Roaring): **~5×** ratio $\rightarrow \mathbf{\sim 100 \text{ TB}}$
- Doc store (titles, URLs, snippet text): $\sim 2 \text{ KB} / \text{page} \times 100B = \mathbf{\sim 200 \text{ TB}}$
- Total per replica: $\sim 300 \text{ TB}$; with 3-way replication $\approx \mathbf{\sim 1 \text{ PB online}}$

Query Throughput

- Sustained: **100,000 QPS**
- Peak (event-driven): **~500,000 QPS** ($\approx 5\times$)
- p99 latency budget: **< 200 ms** from edge to rendered SERP
- Internal budget: 20 ms parse/rewrite + 80 ms shard fan-out + 30 ms rerank + 30 ms snippets + 40 ms slack

Sharding Math

- Per-shard index size target: **~50 GB** in RAM (fits NVMe-backed page cache too)
- Shards per replica: $100 \text{ TB} / 50 \text{ GB} = \mathbf{\sim 2,000 \text{ shards}}$
- Replicas per shard: 3 (HA + read parallelism) $\rightarrow \mathbf{6,000 \text{ index servers}}$
- Fan-out per query: 1 $\rightarrow 2,000$ shards (scatter-gather); coordinator merges top-K

KEY

Key Numbers

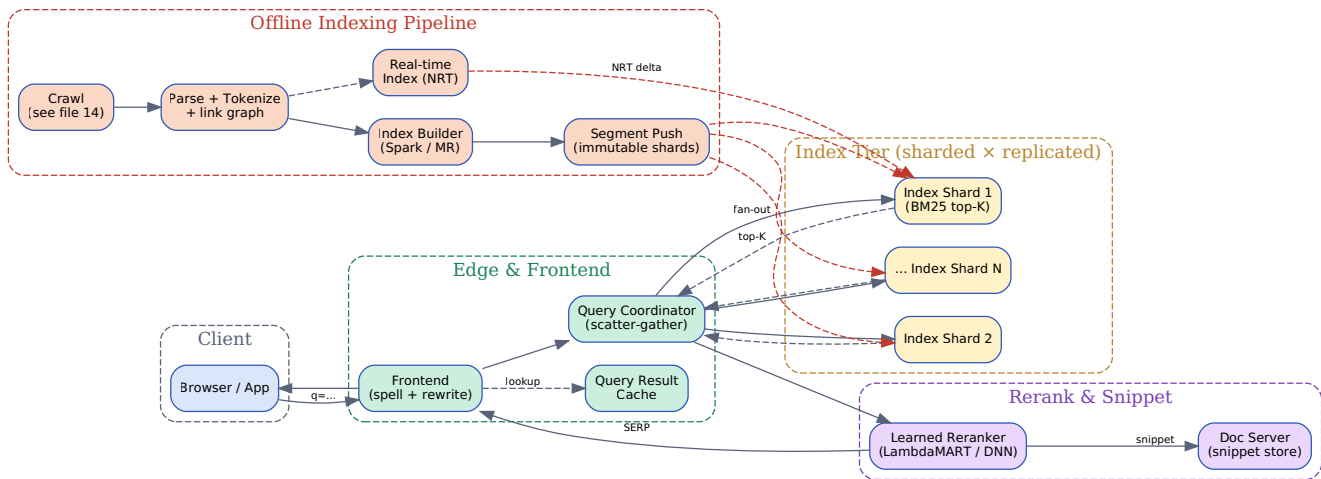
100B pages · 100K QPS sustained / 500K peak · p99 < 200 ms · 100 TB compressed index · $\sim 2,000$ shards $\times$ 3 replicas · $\sim 5 \text{ KB}$ indexed/page.

04

High-Level Architecture

Online serving + offline indexing

Two halves: an **online serving stack** that fans queries out across thousands of index shards and reranks the merged candidates, and an **offline indexing pipeline** that turns crawled pages into immutable index segments and pushes them to serving.



Online path (top): Frontend → Coordinator → Index Servers (sharded) + Doc Servers → Ranker → SERP. Offline path (bottom): Crawl → Parse → Index Build → Index Push.

Online Path

- **Frontend:** auth, spell-correction, query rewrite (synonyms, stemming), result-cache lookup
- **Query Coordinator:** fans the parsed query out to every index shard (root + leaf model)
- **Index Servers:** each shard scores its local docs via BM25 and returns local top-K (e.g., K=1,000)
- **Reranker:** merges shard top-Ks, applies LambdaMART / neural model on top ~200 candidates
- **Doc Server:** separate KV store keyed by doc_id; supplies titles + raw text for snippet generation

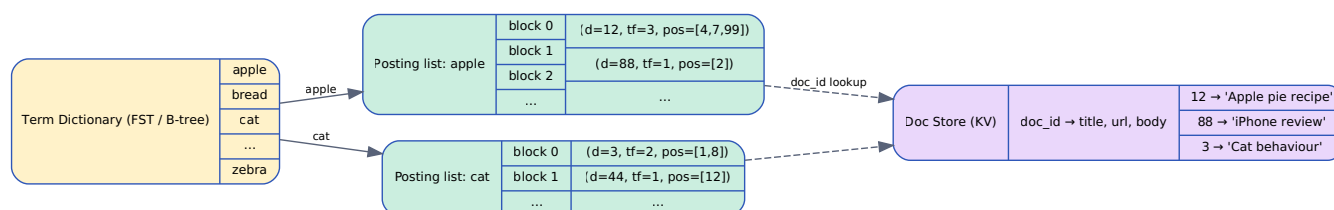
Offline Path

- **Crawler frontier:** upstream (file 14) drops raw HTML into object storage
- **Parser:** extract text, language, links → emit (term, doc_id, tf, positions) tuples
- **Index Builder:** Spark / MapReduce sorts by term, builds posting lists, writes immutable segments
- **Segment Push:** rsync new segments to index servers; hot-reload behind a barrier
- **Real-time index:** small in-memory inverted index for breaking news; merged into base nightly

05 The Inverted Index

Term → posting list

The inverted index is the backbone. For each term in the vocabulary we store a **posting list**: the document IDs that contain the term, plus per-document term frequency and (optionally) positions for phrase queries. Posting lists are sorted by doc_id and block-compressed.



Inverted-index physical layout: term dictionary points into a posting-list file; each list is a sequence of compressed blocks holding doc_ids, term frequencies, and positions.

Posting-List Encoding

```
// Logical record:
// term: "apple"
// df: 42_318_004 // documents containing the term
// postings: [(doc_id, tf, [positions...]), ...] // sorted by doc_id
//
// Physical layout (per posting list):
// header { df, doc_id_min, doc_id_max, num_blocks }
// for each block of 128 postings:
//   doc_ids : delta-encoded then PForDelta-packed
//   tfs : variable-byte (most tfs are tiny)
//   positions : delta-encoded + PForDelta (skipped for non-phrase queries)
//   skip_pointers[] -> { doc_id, file_offset } // every 1,024 docs
//
// Compression ratio: ~5x vs raw int32 doc_ids; ~10x with positions skipped.
// Hot heads (small df_inverse, i.e. common terms) live in RAM; tails on SSD.
```

Tiered Storage

Tier	Resident	Latency	What lives here
RAM	~10% of index	< 1 ms	Heads of common terms, term dict, skip pointers
NVMe SSD	~70% of index	~100 µs	Bulk posting lists, doc store hot shard
HDD / object store	~20% (cold)	~10 ms	Long-tail postings, archive doc bodies

INSIGHT

Why immutable segments?

Posting lists are append-mostly and read-heavy. Immutable segments + background merges (LSM-style) let us keep posting lists densely packed, memory-map them, and avoid per-document write amplification.

Lucene/Elasticsearch use exactly this pattern.

06

Sharding Strategy

Document-partition vs term-partition

Two ways to slice a 100 TB index across 2,000 machines: by **document** (every shard owns ~50M docs and holds the full vocabulary for those docs) or by **term** (every shard owns a slice of the vocabulary and holds the global posting list for those terms). The trade-off shapes the entire serving stack.

Aspect	Doc-partition (recommended)	Term-partition
Query fan-out	All shards (scatter-gather)	Only shards owning query terms (1–5 typical)
Network	High: $O(N_{\text{shards}})$ RPCs / query	Low: $O(\text{query_terms})$ RPCs
Hotspot risk	Even (random doc_id placement)	Severe — common terms (“the”) hammer one shard
Indexing	Embarrassingly parallel by doc	Requires global sort by term; cross-machine shuffle
Adding docs	Append to one shard	Touches every shard owning a term in the doc
Replica granularity	Per shard (simple)	Per term range (uneven sizes)
Recovery	Lose 1/N of corpus per shard down → degrade	Lose entire term ranges → wrong answers
Used by	Google, Bing, Elasticsearch, Solr	Mostly academic / specialised IR systems

Why Doc-Partition Wins at Web Scale

- **Hotspot resistance:** hash(doc_id) spreads load uniformly; query 'the' hits all shards equally
- **Fault tolerance:** a dead shard means missing 1/N of candidates, not missing a whole term
- **Throughput:** 2,000 shards × 3 replicas = 6,000 cores doing scoring in parallel
- **Indexing speed:** each builder writes its own segments, no cross-cluster shuffle

Cost of Scatter-Gather

- Every query talks to every shard → tail-latency amplification (slowest shard wins)
- Mitigation: **request hedging** — re-issue to a replica when p95 elapses
- Mitigation: **tiered serving** — high-quality shard tier handles 90% of queries; full corpus only when shallow tier returns < threshold candidates

TIP**Recommendation**

Doc-partition with hash(doc_id) sharding and 3-way replication. Pair with a hot/cold tier so most queries are answered by a smaller, faster subset of the corpus, falling back to full fan-out only when needed.

07

Query Pipeline

From query string to SERP

End-to-End Steps

1. **Receive** query at Frontend; record query_id, user locale, session
2. **Normalize**: lowercase, Unicode NFKC, strip punctuation
3. **Spell-correct**: noisy-channel model + click-log priors → 'did you mean'
4. **Tokenize** with the same analyzer used at index time
5. **Rewrite**: stem, expand synonyms, drop stopwords (or downweight)
6. **Cache check**: hash(rewritten_query, locale) → query result cache
7. **Coordinator fan-out**: send to all index shards (or hot tier first)
8. **Per-shard scoring**: BM25 over posting lists; emit local top-K ($K \approx 1,000$)
9. **Merge** at coordinator: priority queue keyed by score → global top-N ($N \approx 200$)
10. **Rerank**: learned-to-rank model with rich features (PageRank, freshness, click-through)
11. **Snippet**: re-fetch doc bodies, highlight matched terms
12. **Render SERP**: 10 results + ads + features (KG card, images)
13. **Log**: query, results, click position → feedback loop for ranking model

Latency Budget

Stage	Budget	Notes
Edge + Frontend	20 ms	TLS, parse, spell-correct, cache lookup
Coordinator fan-out	80 ms	Slowest-shard latency; hedged requests
Per-shard scoring	50 ms (parallel)	Decompress postings + BM25 + heap top-K
Merge top-K	5 ms	Priority queue across N shards
Rerank	30 ms	LambdaMART or small DNN over ~200 docs
Snippet generation	30 ms	Doc-server fetch + highlight
Render + return	20 ms	HTML / JSON response
Total p99	< 200 ms	Overlapping where possible

INSIGHT

Tail latency hedging

At 2,000 shards a query is only as fast as its slowest shard. If each shard has $p99 = 50$ ms, the fan-out $p99$ is much worse. Mitigation: send the request to one replica, and after 30 ms fire a tied request to a second replica; take whichever returns first. Recovers most of the tail.

08

Ranking

BM25, link signals, learned-to-rank

Two-Stage Ranking

- **Candidate generation (recall):** BM25 per shard, top 1,000 each
- **Reranking (precision):** learned model over top ~200 globally with rich features
- Why two stages? BM25 is cheap (millions of docs/sec/core); LambdaMART is expensive but only runs on hundreds

BM25 Scoring

```
# BM25 score for query Q against document D
# Constants: k1 ≈ 1.2, b ≈ 0.75
#   tf(t,D)  = term frequency of t in D
#   df(t)    = number of docs containing t
#   |D|      = length of D in tokens
#   avgdl    = mean doc length in the corpus
#   N        = total docs in the corpus

def bm25(Q, D, stats, k1=1.2, b=0.75):
    score = 0.0
    for t in Q:
        if t not in D: continue
        idf = log((stats.N - stats.df[t] + 0.5) /
                  (stats.df[t] + 0.5) + 1)
        tf  = D.tf[t]
        norm = 1 - b + b * (D.length / stats.avgdl)
        score += idf * (tf * (k1 + 1)) / (tf + k1 * norm)
    return score
```

Top-K Heap Merge Across Shards

```
# Coordinator merges per-shard top-K into a global top-N ( $N \leq K$ )
import heapq

def merge_topk(shard_results, N=200):
    # shard_results: list[ list[(score, doc_id, shard_id)] ] per-shard sorted desc
    heap = [] # min-heap of size N
    for shard in shard_results:
        for score, doc_id, sid in shard:
            if len(heap) < N:
                heapq.heappush(heap, (score, doc_id, sid))
```

```

elif score > heap[0][0]:
    heapq.heapreplace(heap, (score, doc_id, sid))
else:
    break # shard list is sorted; safe to stop
return sorted(heap, key=lambda x: -x[0])

```

Ranking Signal Categories

Category	Examples	Signal type
Text relevance	BM25, term proximity, exact-phrase match	Per-query × per-doc
Document quality	PageRank, link graph, spam score, content length	Per-doc (offline)
Freshness	doc_age, last-modified, crawl frequency	Per-doc (offline) + RT
User behaviour	Click-through rate, dwell time, query reformulation	Aggregated from logs
Personalisation	Locale, language, recent queries, location	Per-query × per-user
Authority	Domain trust, HTTPS, author signals	Per-doc (offline)
Intent	Navigational vs informational classifier	Per-query

Learned-to-Rank Reranker

- **Model:** LambdaMART (gradient-boosted trees) — workhorse since ~2010; or a small Transformer cross-encoder for top-50
- **Training:** pairwise loss on labelled (query, doc, relevance) judgements + click logs
- **Features per (q, d):** ~hundreds (BM25 of body/title/anchor, PageRank, freshness, click features, language match)
- **Inference:** ~30 ms for 200 candidates × 500 trees on a single CPU; SIMD-friendly
- **A/B:** every model change rolled out behind interleaved experiments measured by NDCG@10 + click-through

INSIGHT

PageRank in 2026

Link graph signals (PageRank-style eigenvector centrality) are still computed offline over a graph of ~1T edges. They survive as one feature among hundreds in the LTR model — not the dominant signal Brin and Page started with, but still a strong prior for authority.

09

Indexing Pipeline

Crawl → segments → serving

Daily Batch Build

1. **Input:** raw crawl corpus in object storage (Parquet of [url, html, fetched_at])
2. **Parse stage:** Spark job extracts text, language, links, metadata
3. **Tokenize:** per-language analyzer; emit (doc_id, term, tf, positions)
4. **Sort + group:** shuffle by (shard_id, term); each shard sees its slice
5. **Build segments:** per shard, write immutable Lucene-style segment files
6. **Push:** upload to per-shard staging area; index server downloads
7. **Atomic swap:** behind a barrier, all replicas of a shard cut over to new generation

Real-Time Insert Path

- **Why?** News, sports, social trends — must be searchable in minutes, not 24 hours
- **Write path:** Parser → Kafka topic doc_updates (key = doc_id)
- **NRT indexer:** consumes Kafka, builds an in-memory mini-index per shard (~GB)
- **Merge:** nightly batch absorbs the NRT delta into the main segment
- **Serving:** queries hit BOTH the main index and the NRT delta; scores combined

Freshness vs Build Cost

Strategy	Freshness	Build cost	Where used
Full daily rebuild	24 h	Highest (re-process 100B docs)	Long-tail content
Incremental segments + nightly merge	Hours	Medium	Most content
Real-time mini-index (NRT)	Minutes	Low (in-memory only)	News, trending
Push-on-publish	Seconds	Per-doc network cost	Featured publishers, AMP

TIP

Three-tier freshness model

Run all three: nightly full rebuild for the long tail, incremental merges for hourly updates, and an NRT in-memory index for trending content. Queries hit all tiers; scores combined. Keeps cost bounded while staying fresh on what users actually care about right now.

10

Caching Layers

Result, posting-list, and document caches

Why caching matters here

- Query distribution is heavily Zipfian: top ~1% of queries cover ~50% of volume
- Recomputing the SERP for 'weather' or 'youtube' 100K times/sec is wasteful
- Posting lists for common terms ('the', 'and') are huge and accessed by every query

Layer	Key	Value	Size	Hit rate	TTL
Query result cache	hash(query, locale)	Rendered SERP (top-10)	~100 GB	30–50%	1–10 min
Posting-list cache	term_id	Decompressed posting head	~5 GB / shard	~80% on heads	Until segment swap
Doc fragment cache	doc_id	Title + snippet body	~50 GB / region	~60%	Hours
Spell / rewrite cache	raw query	Rewritten query	~10 GB	~70%	1 day

Query Result Cache Details

- **Eligibility:** only non-personalised, non-localised slices cached; logged-in users still benefit from sub-feature caches
- **Invalidation:** short TTL + explicit purge on big news events (e.g., 'super bowl 2026' result needs refreshing live)
- **Storage:** Memcached / Redis cluster geographically replicated; ~200 nodes
- **Win:** a 30% hit rate at 100K QPS removes 30K QPS from the index tier — saves billions in serving cost

Posting-List Cache Details

- Each index server keeps decompressed posting-list **heads** (first few blocks) for the most-used terms
- Eviction: LFU; resets after segment swap
- Saves the per-query decompression cost on common terms — biggest CPU win in the index server

WARNING

Personalisation vs cache hit

Every personal feature you add (location, recent history, account preferences) shrinks the cache key space and tanks the result-cache hit rate. Keep personalisation as a final reranking step over cached candidates, not a first-class cache key.

11

Doc Store & Snippet Generation

Showing why a result matters

Doc Store

- **Purpose:** serve the data needed to render a result row — title, URL, body excerpt
- **Schema:** `doc_id` → { `url`, `title`, `body_text`, `lang`, `last_modified` }
- **Backing store:** Bigtable / RocksDB / custom SSTable; sharded by `doc_id`
- **Size:** ~2 KB / doc × 100B = ~200 TB; replicated 3×
- **Access pattern:** 200 random `doc_id` reads per query (one per candidate)

Snippet Algorithm

1. Fetch doc body for top-10 reranked results (parallelised across doc-store shards)
2. Slide a fixed-width window (≈ 30 tokens) over the body
3. Score each window by sum of query-term occurrences + IDF weight
4. Pick the highest-scoring window; truncate at sentence boundary
5. Bold the matched query terms; HTML-escape; return ~160-char snippet

Snippet Cache

- Cache key: (`doc_id`, `query_terms_hash`)
- TTL: 1 day; invalidated when doc body changes
- Hit rate: ~60% (popular doc + popular query combos)
- Without this cache, snippet generation can dominate the latency budget

INSIGHT

Why a separate doc store?

We never store the full doc body inside posting-list shards — it would balloon the inverted index for no scoring benefit. Splitting the index server (scoring, latency-critical, RAM-bound) from the doc server (snippet, throughput-bound, SSD-bound) lets each scale independently.

12

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
1 of 3 shard replicas down	Query still served by remaining 2	Health-check / heartbeat	Auto-promote spare replica; rebuild from segments in object storage
Entire shard (all replicas) down	Missing 1/N of candidates → degraded recall	Coordinator timeout %	Serve from N-1 shards; flag SERP as 'partial'; alert SREs
Coordinator overloaded	Tail latency blows past 200 ms	p99 alarm	Shed load (drop low-priority queries); auto-scale coordinator pool
Index push corrupt segment	Wrong / missing results	Checksum + canary query suite	Refuse swap; revert to previous generation
Reranker model OOM / crash	BM25-only ranking → noticeable quality drop	Model-server health	Fall back to BM25 ordering; alert; roll model back
Doc store shard down	Cannot generate snippets for some results	Doc-server timeout	Show title + URL only; fall back to cached snippet
NRT pipeline lag	Trending content stale	Kafka lag metric	Alert; scale NRT consumers; queries see slightly older content
Datacenter loss	Region offline	Edge health checks	Geo-failover to surviving DC; serve from cached SERPs first

Graceful Degradation Ladder

1. Healthy: full fan-out + LTR rerank + personalisation + snippets
2. Mild: skip personalisation reranker; keep core LTR
3. Worse: skip LTR; sort merged results by BM25
4. Severe: serve from query result cache only
5. Critical: return cached SERPs from CDN edge with 'service degraded' banner

WARNING

Recall degrades silently

When a shard is down, the SERP looks normal — it's just missing some good candidates. Always emit a per-query metric for the number of shards that responded; alert when it drops below the configured floor.

13

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Sharding axis	Document-partition	Scatter-gather every query but no hotspots and easy HA. Term-partition is cheaper per query but suffers severe load skew on common terms.
Index format	Immutable segments + LSM merges	Background merge cost; simpler reads, dense compression, mmap-friendly. In-place updates would write-amplify badly.
Compression	PForDelta / Roaring + var-byte tf	Slightly more CPU on read, ~5× smaller index. Pure raw int32 would explode RAM cost.
Two-stage ranking	BM25 → LTR rerank top-200	BM25 is recall-imperfect but cheap; LTR is precise but expensive — only run it on the shortlist.
Freshness	Daily batch + NRT delta	Trending content fresh in minutes; long tail rebuilt nightly. Per-doc push-on-publish is fresher but unaffordable for 100B docs.
Personalisation	Final rerank only	Preserves cache hit rate. Personalising candidate gen would shrink shareable cache to near-zero.
Cache TTL	Short (1–10 min) on results	Slight staleness during news events; very high freshness elsewhere. Long TTL serves stale SERPs in fast-moving topics.
Replication	3× per shard	Lose any 2 nodes per shard → still serving. 2× would be cheaper but cuts safety margin in half.
Tail-latency hedging	Tied requests at p95	~5–10% extra fan-out load; recovers most of the long tail. Without it, p99 dominated by slowest replica.

14

Scalability & Operations

Running the fleet

Capacity Planning

- **Index tier:** ~6,000 servers (2,000 shards × 3 replicas); ~50 GB index each, 256 GB RAM, NVMe
- **Doc tier:** ~3,000 servers; SSD-heavy; ~100 TB / replica × 3
- **Coordinators:** ~500 stateless boxes; auto-scale on QPS
- **Reranker:** ~1,000 GPU/CPU servers; model held entirely in RAM
- **Cache tier:** ~500 Redis/Memcached nodes per region

Deployment Cadence

- **Index segments:** pushed continuously; replicas rolled one at a time per shard
- **Coordinator / frontend:** blue-green deploys behind LB; canary 1% → 10% → 100%
- **Ranking model:** shadow traffic → interleaved A/B → full rollout if NDCG@10 wins
- **Schema changes:** double-write old + new posting list format until full reindex done

Observability

- **Per-query metrics:** latency p50/p99, shards responded, cache hit, ranker stage timings
- **Quality metrics:** NDCG@10 on a labelled probe set, CTR@1, query reformulation rate
- **Index health:** segment count, merge backlog, NRT lag, push success rate
- **Capacity:** RAM headroom per shard, SSD IOPS, coordinator CPU

Cost Levers

Lever	Effect	Risk
Shrink ranker top-200 → top-100	~40% reranker cost	Slight NDCG loss
Cold-tier more long-tail postings to disk	Less RAM per shard	Higher tail latency
Tighter result-cache eligibility	Higher hit rate	Less personalisation
Fewer replicas (3 → 2)	33% fewer servers	No rolling deploy + repair simultaneously

15

Interview Playbook

How to present this

Web search is a depth-of-knowledge interview: the surface area is huge (crawl, index, rank, serve, ML, ops), so signal which area you want to go deep on early. The strongest candidates pick **one** of (a) inverted-index internals, (b) sharding/serving, or (c) ranking, and own it convincingly while keeping the others at the right altitude.

45-Minute Interview Outline

1. **Clarify (3 min):** 100B pages, 100K QPS, p99 < 200 ms, daily + NRT freshness, crawl is solved
2. **Capacity (5 min):** 5 KB/page → 500 TB raw → 100 TB compressed; 2,000 shards × 3 replicas
3. **High-level arch (5 min):** two halves — online serve + offline build; draw the diagram
4. **Inverted index (8 min):** term → posting list (doc_id, tf, positions); block compression; tiered storage
5. **Sharding (5 min):** doc-partition vs term-partition; pick doc; explain hotspot argument
6. **Query path (5 min):** rewrite → fan-out → BM25 top-K → merge → LTR rerank → snippet
7. **Ranking (8 min):** two stages, BM25 formula, LTR features, PageRank as one signal
8. **Caching + freshness (3 min):** result cache, NRT path, three-tier freshness model
9. **Failures + trade-offs (3 min):** shard down → degrade, hedging, doc-partition rationale

Key Talking Points

- “100B pages, ~100 TB compressed inverted index across 2,000 shards × 3 replicas” — sets scale
- “Doc-partition because hash(doc_id) avoids the 'the' hotspot” — answers the obvious follow-up
- “Two-stage ranking: BM25 for recall, LambdaMART for precision” — shows ML literacy
- “Hedged requests at p95 to tame scatter-gather tail latency” — distributed-systems depth
- “Three-tier freshness: nightly + incremental + NRT” — handles 'how do you index breaking news?'
- “Snippet generation is a separate doc-store hop” — shows you've thought past ranking
- “Personalisation is final rerank, not candidate gen, to preserve cache hit rate” — non-obvious

Common Follow-ups & Answers

- **Q: Why not term-partition?** A: Severe hotspots on common terms; one shard handling every 'the' query melts. Doc-partition spreads load uniformly via hash(doc_id), at the cost of fanning out every query.
- **Q: How do you handle phrase queries?** A: Posting lists store positions; intersect doc_id sets and check position adjacency. Expensive, so we keep positions cached separately and skip them when not needed.

- **Q: How fresh can you be?** A: NRT mini-index in RAM consumes Kafka of new docs and serves alongside the main index — fresh in minutes. Nightly batch absorbs the delta into base segments.
- **Q: How do you control p99 with 2,000 shards?** A: Tied / hedged requests after p95, tiered serving (hot tier covers 90% of queries), per-shard request budgets with shed-on-overload.
- **Q: How do you measure ranking quality?** A: NDCG@10 on a labelled probe set + interleaved live experiments measuring CTR and query-reformulation rate. Every model rollout is A/B'd.
- **Q: How do you handle a shard going down?** A: Two replicas keep serving. If all three die, coordinator returns N-1 results and flags the SERP partial; SREs page; rebuild from object-storage segments.

KEY

Memorize These Numbers

100B pages · 100K QPS sustained / 500K peak · p99 < 200 ms · 5 KB indexed / page · 500 TB raw → 100 TB compressed · 2,000 shards × 3 replicas · BM25 ($k_1=1.2$, $b=0.75$) · rerank top-200 with LambdaMART · 30–50% query-cache hit rate.